

Laboratorio: Diseño e Implementación de Objetos de Base de Datos con SQL

Este documento recoge el desarrollo paso a paso del laboratorio oficial de Microsoft Learning para el diseño e implementación de objetos de base de datos en SQL Server.

El objetivo del ejercicio es construir el esquema de datos para una plataforma de comercio electrónico (EcommerceDB). A lo largo de la práctica se implementan tablas estándar con restricciones de integridad, tablas temporales para auditar cambios de precios, almacenamiento nativo de metadatos en formato JSON, particionamiento físico para gestionar grandes volúmenes de pedidos y secuencias numéricas independientes.

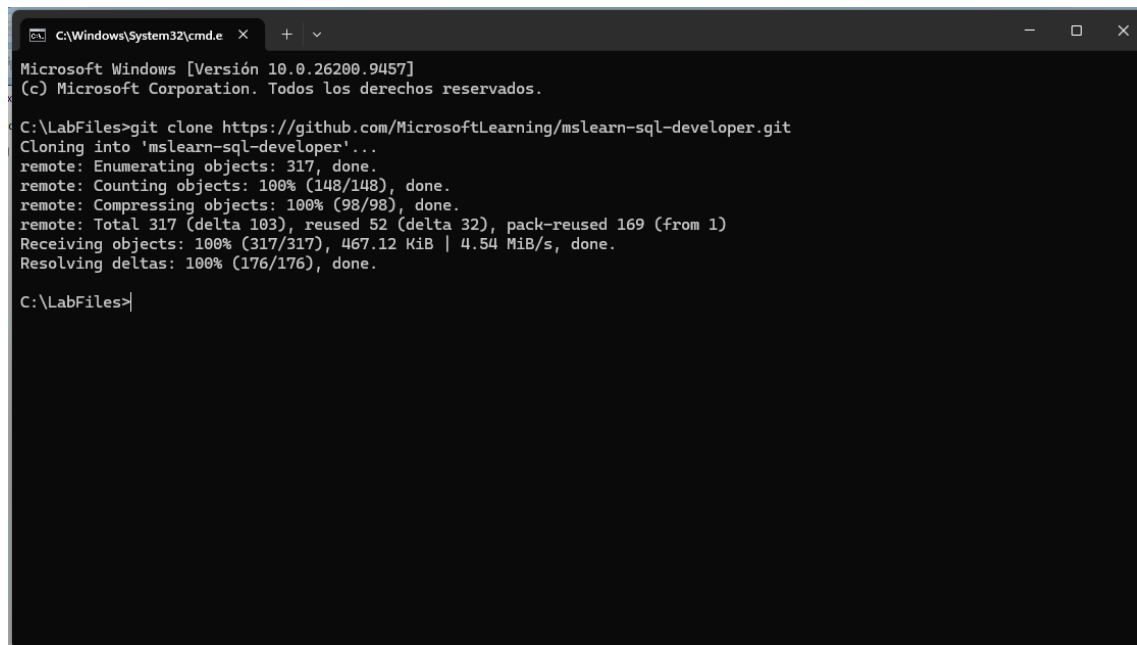
Información del Entorno

- Sistema gestor: Microsoft SQL Server 2025 (v17.0) o superior
- Herramienta de gestión: SQL Server Management Studio (SSMS)
- Base de datos: EcommerceDB
- Repositorio base: [MicrosoftLearning/mslearn-sql-developer](https://github.com/MicrosoftLearning/mslearn-sql-developer)

1. Configuración del Entorno (Setup environment)

Para contar con los scripts y recursos del laboratorio, abrimos una terminal y clonamos el repositorio en el directorio de trabajo local C:\LabFiles .

```
cd C:\LabFiles
git clone https://github.com/MicrosoftLearning/mslearn-sql-developer.git
```



```
C:\Windows\System32\cmd.exe
Microsoft Windows [Versión 10.0.26200.9457]
(c) Microsoft Corporation. Todos los derechos reservados.

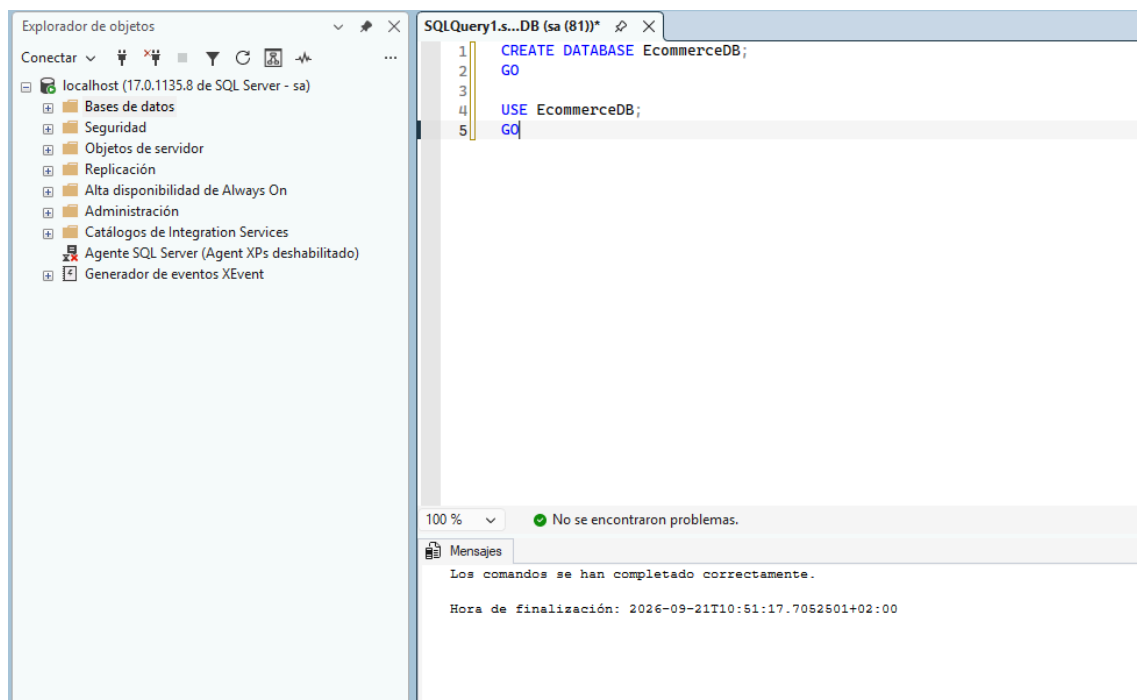
C:\LabFiles>git clone https://github.com/MicrosoftLearning/mslearn-sql-developer.git
Cloning into 'mslearn-sql-developer'...
remote: Enumerating objects: 317, done.
remote: Counting objects: 100% (148/148), done.
remote: Compressing objects: 100% (98/98), done.
remote: Total 317 (delta 103), reused 52 (delta 32), pack-reused 169 (from 1)
Receiving objects: 100% (317/317), 467.12 KiB | 4.54 MiB/s, done.
Resolving deltas: 100% (176/176), done.

C:\LabFiles>
```

2. Creación de la Base de Datos (Create a new database)

Iniciamos sesión en SQL Server Management Studio (SSMS) conectándonos a la instancia local. Abrimos una nueva ventana de consulta y creamos la base de datos `EcommerceDB`.

```
CREATE DATABASE EcommerceDB;  
GO  
  
USE EcommerceDB;  
GO
```



Al ejecutar la consulta, el panel de mensajes confirma que la operación finalizó con éxito.

3. Tablas Principales con Restricciones (Create core tables with constraints)

Construimos las tablas fundamentales de la tienda: proveedores (`Supplier`), categorías (`Category`) y productos (`Product`).

En este paso aplicamos reglas de integridad en el propio motor:

- Claves primarias autoincrementales con `IDENTITY(1,1)`.
- Restricciones de unicidad (`UNIQUE`) en nombres de proveedores y categorías.
- Restricciones `CHECK` para garantizar que el precio sea mayor que 0 (`BasePrice > 0`) y que el stock no sea negativo (`StockQuantity >= 0`).
- Claves foráneas (`FOREIGN KEY`) para mantener la integridad referencial.

- Índices no agrupados sobre las columnas foráneas (`IX_Category` , `IX_Supplier`) para optimizar futuras consultas y uniones.

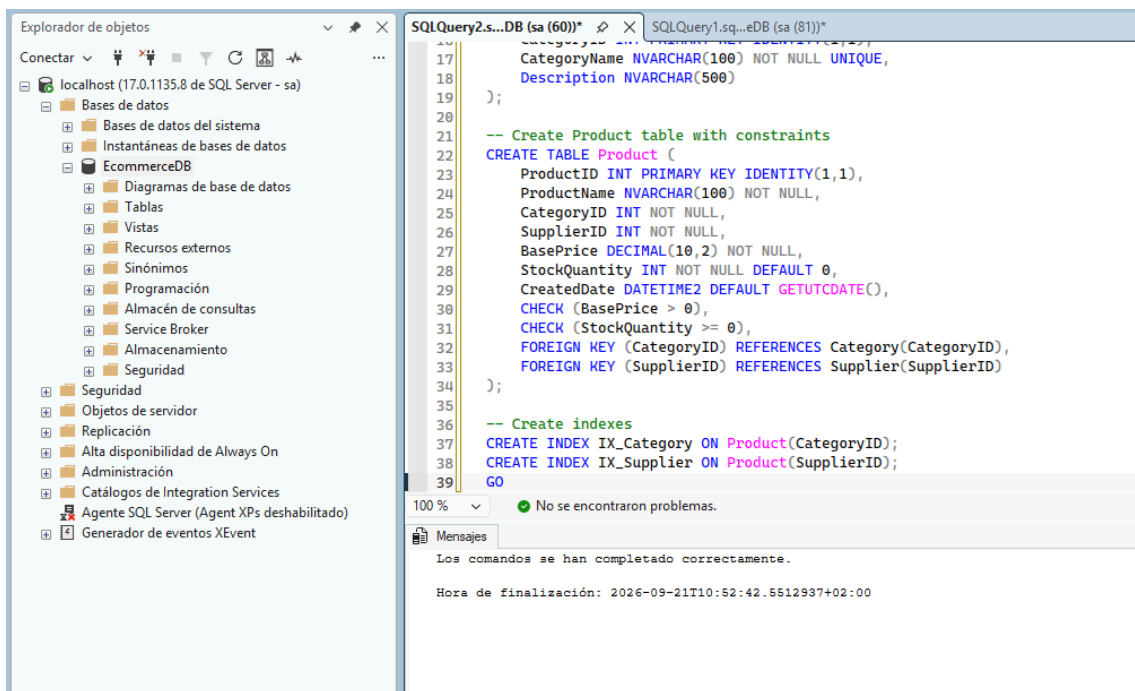
```
USE EcommerceDB;
GO

-- Tabla Supplier
CREATE TABLE Supplier (
    SupplierID INT PRIMARY KEY IDENTITY(1,1),
    SupplierName NVARCHAR(100) NOT NULL UNIQUE,
    Country NVARCHAR(50) NOT NULL,
    Email NVARCHAR(100),
    Phone NVARCHAR(20),
    CreatedDate DATETIME2 DEFAULT GETUTCDATE()
);

-- Tabla Category
CREATE TABLE Category (
    CategoryID INT PRIMARY KEY IDENTITY(1,1),
    CategoryName NVARCHAR(100) NOT NULL UNIQUE,
    Description NVARCHAR(500)
);

-- Tabla Product con restricciones
CREATE TABLE Product (
    ProductID INT PRIMARY KEY IDENTITY(1,1),
    ProductName NVARCHAR(100) NOT NULL,
    CategoryID INT NOT NULL,
    SupplierID INT NOT NULL,
    BasePrice DECIMAL(10,2) NOT NULL,
    StockQuantity INT NOT NULL DEFAULT 0,
    CreatedDate DATETIME2 DEFAULT GETUTCDATE(),
    CHECK (BasePrice > 0),
    CHECK (StockQuantity >= 0),
    FOREIGN KEY (CategoryID) REFERENCES Category(CategoryID),
    FOREIGN KEY (SupplierID) REFERENCES Supplier(SupplierID)
);

-- Creación de índices
CREATE INDEX IX_Category ON Product(CategoryID);
CREATE INDEX IX_Supplier ON Product(SupplierID);
GO
```



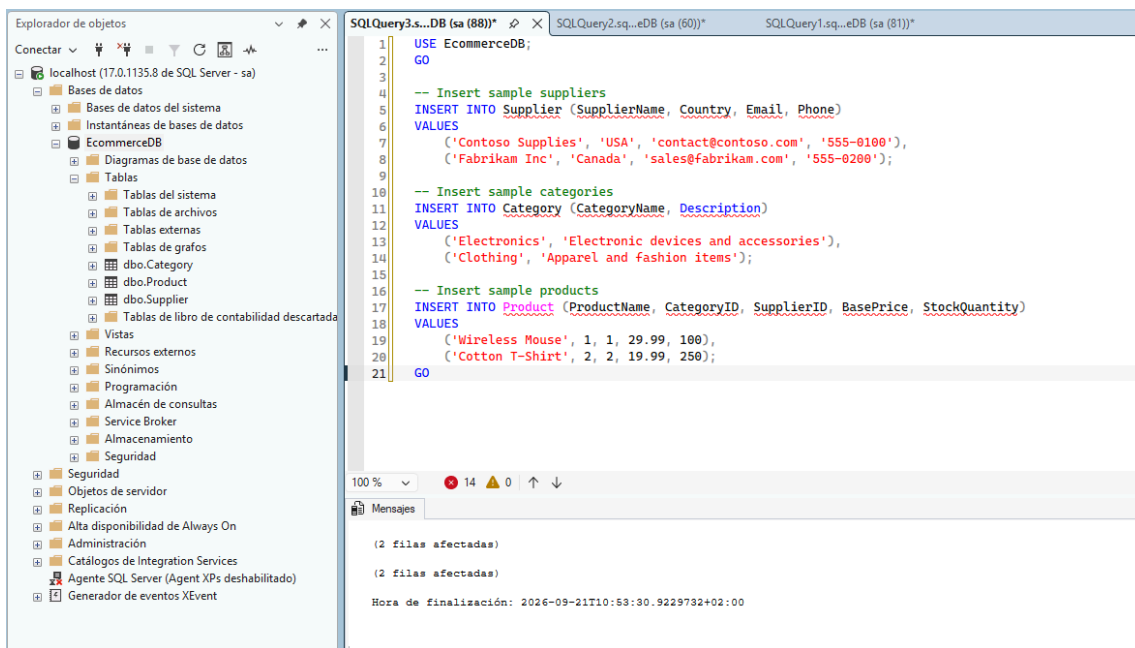
A continuación, insertamos registros de prueba para validar la estructura:

```
USE EcommerceDB;
GO

-- Proveedores de ejemplo
INSERT INTO Supplier (SupplierName, Country, Email, Phone)
VALUES ('Contoso Supplies', 'USA', 'contact@contoso.com', '555-0100'),
       ('Fabrikam Inc', 'Canada', 'sales@fabrikam.com', '555-0200');

-- Categorías de ejemplo
INSERT INTO Category (CategoryName, Description)
VALUES ('Electronics', 'Electronic devices and accessories'),
       ('Clothing', 'Apparel and fashion items');

-- Productos de ejemplo
INSERT INTO Product (ProductName, CategoryID, SupplierID, BasePrice, StockQuantity)
VALUES ('Wireless Mouse', 1, 1, 29.99, 100),
       ('Cotton T-Shirt', 2, 2, 19.99, 250);
GO
```



4. Tabla Temporal para Historial de Precios (Create a temporal table for price history)

Las tablas temporales con versionado del sistema permiten mantener una pista de auditoría completa de los cambios de datos sin necesidad de programar disparadores manuales.

Creamos la tabla `ProductPrice` con `SYSTEM_VERSIONING = ON` y dos columnas ocultas de fecha (`SysStartTime` y `SysEndTime`). Luego cargamos precios iniciales y realizamos una actualización sobre el producto 1 para generar el registro histórico.

```
USE EcommerceDB;
GO

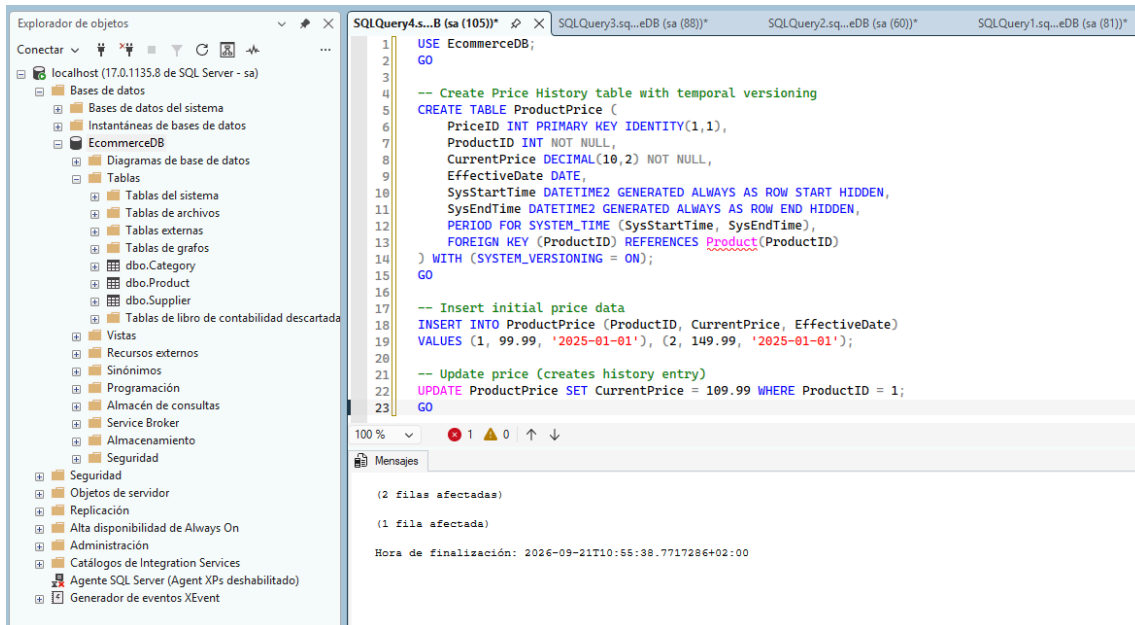
-- Creación de tabla temporal con control de versiones
CREATE TABLE ProductPrice (
    PriceID INT PRIMARY KEY IDENTITY(1,1),
    ProductID INT NOT NULL,
    CurrentPrice DECIMAL(10,2) NOT NULL,
    EffectiveDate DATE,
    SysStartTime DATETIME2 GENERATED ALWAYS AS ROW START HIDDEN,
    SysEndTime DATETIME2 GENERATED ALWAYS AS ROW END HIDDEN,
    PERIOD FOR SYSTEM_TIME (SysStartTime, SysEndTime),
    FOREIGN KEY (ProductID) REFERENCES Product(ProductID)
) WITH (SYSTEM_VERSIONING = ON);
GO

-- Inserción de precios iniciales
INSERT INTO ProductPrice (ProductID, CurrentPrice, EffectiveDate)
VALUES (1, 99.99, '2025-01-01');
```

```
(2, 149.99, '2025-01-01');
```

-- Actualización de precio (genera la entrada histórica automáticamente)

```
UPDATE ProductPrice
SET CurrentPrice = 109.99
WHERE ProductID = 1;
GO
```



Para consultar la evolución temporal completa del producto, utilizamos la cláusula `FOR SYSTEM_TIME ALL` :

```
USE EcommerceDB;
GO

SELECT ProductID, CurrentPrice, SysStartTime, SysEndTime
FROM ProductPrice
FOR SYSTEM_TIME ALL
WHERE ProductID = 1;
```

The screenshot shows the SQL Server Enterprise Manager interface. On the left, the 'Object Explorer' pane displays the database structure for 'EcommerceDB', including tables like 'Product' and 'ProductPrice'. The main window shows a SQL query in the 'SQL Query5.s...B (sa (107))' window. The query is as follows:

```
1 USE EcommerceDB;
2 GO
3
4 -- Query price history
5 SELECT ProductID, CurrentPrice, SysStartTime, SysEndTime
6 FROM ProductPrice
7 FOR SYSTEM_TIME ALL
8 WHERE ProductID = 1;
```

Below the query window, the 'Results' pane shows the output of the query. It contains a table with 4 columns: 'ProductID', 'CurrentPrice', 'SysStartTime', and 'SysEndTime'. There are two rows of data:

ProductID	CurrentPrice	SysStartTime	SysEndTime
1	109.99	2026-09-21 08:55:38.7666951	9999-12-31 23:59:59.9999999
1	99.99	2026-09-21 08:55:38.7657312	2026-09-21 08:55:38.7666951

El resultado muestra tanto el precio vigente actual (109.99) como el precio previo (99.99) con sus respectivos intervalos temporales.

5. Columnas JSON para Metadatos (Add JSON columns for metadata)

Para almacenar atributos variables que dependen del tipo de producto (como talla, color o material) sin tener que crear múltiples columnas vacías, incorporamos una columna de tipo nativo `JSON`.

Para que las búsquedas por estos atributos sean ágiles, creamos una columna calculada persistente basada en la función `JSON_VALUE` e indexamos dicha columna:

```
USE EcommerceDB;
GO

-- Añadimos la columna nativa JSON
ALTER TABLE Product ADD Metadata JSON;
GO

-- Columna calculada para extraer el color
ALTER TABLE Product ADD MetadataColor AS JSON_VALUE(Metadata, '$.color');
GO

-- Índice sobre la columna calculada
CREATE NONCLUSTERED INDEX IX_Product_Metadata_Color
ON Product (MetadataColor);
GO

-- Actualización de productos con metadatos JSON
UPDATE Product
SET Metadata = N'{"color":"blue","size":"large","material":"cotton"}'
```

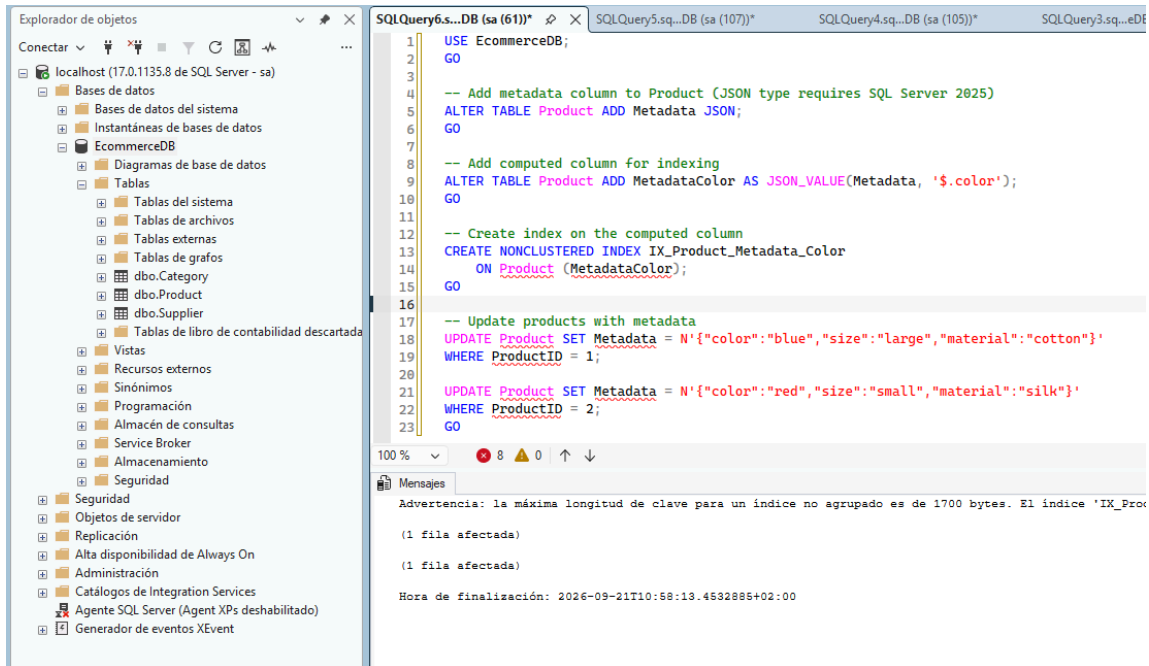
```
WHERE ProductID = 1;
```

```
UPDATE Product
```

```
SET Metadata = N'{"color":"red","size":"small","material":"silk"}'
```

```
WHERE ProductID = 2;
```

```
GO
```

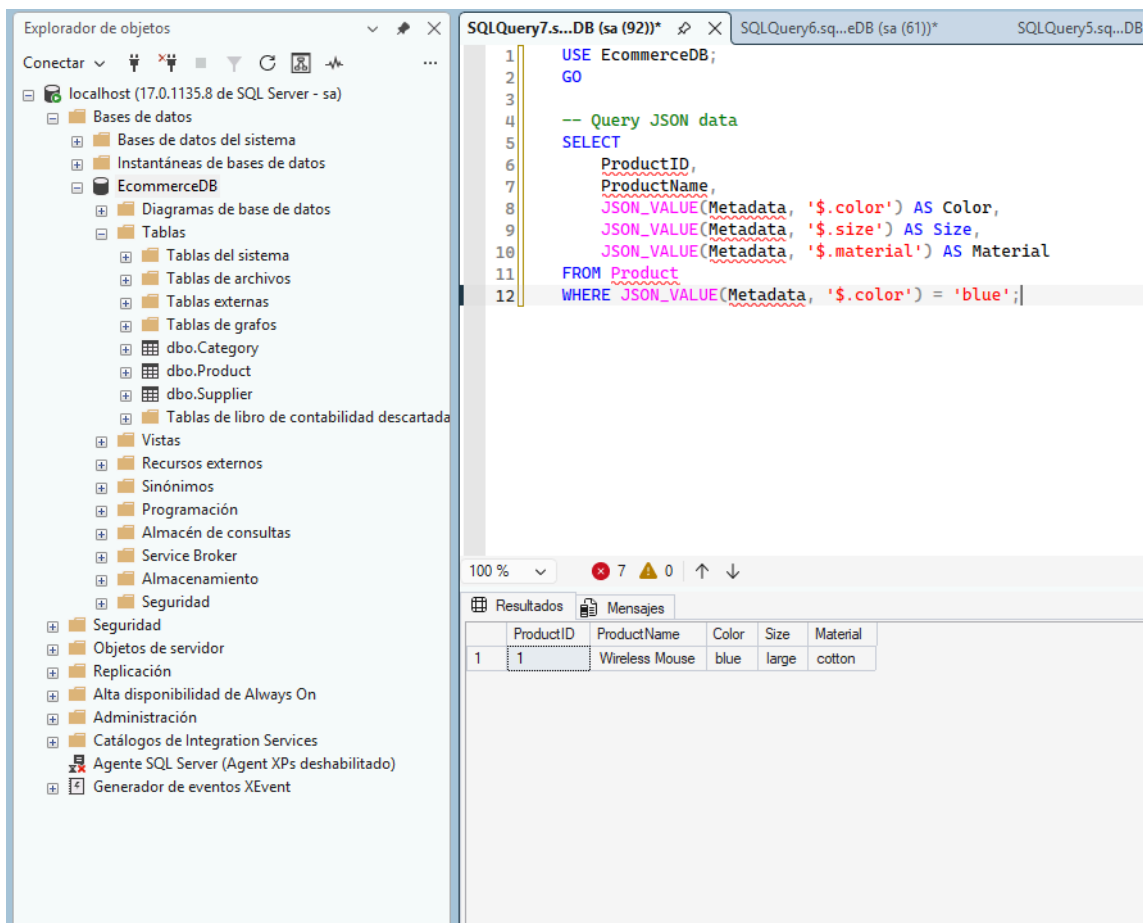


Consultamos los datos leyendo directamente las propiedades del documento JSON con `JSON_VALUE` :

```
USE EcommerceDB;
```

```
GO
```

```
SELECT ProductID, ProductName,
       JSON_VALUE(Metadata, '$.color') AS Color,
       JSON_VALUE(Metadata, '$.size') AS Size,
       JSON_VALUE(Metadata, '$.material') AS Material
FROM Product
WHERE JSON_VALUE(Metadata, '$.color') = 'blue';
```



6. Tabla de Pedidos Particionada (Create a partitioned order table)

El particionamiento divide físicamente tablas voluminosas en partes más pequeñas para acelerar las búsquedas por fecha y simplificar tareas de mantenimiento.

1. Creamos una función de partición por rangos (PF_OrderDate) basada en trimestres del año 2025.
2. Definimos el esquema de partición (PS_OrderDate) asignado al grupo de archivos primario.
3. Creamos la tabla [Order] vinculada a este esquema, incluyendo OrderDate en la clave primaria para la correcta alineación del índice agrupado.
4. Añadimos un índice no agrupado particionado e insertamos pedidos en distintas fechas.

```
USE EcommerceDB;
GO

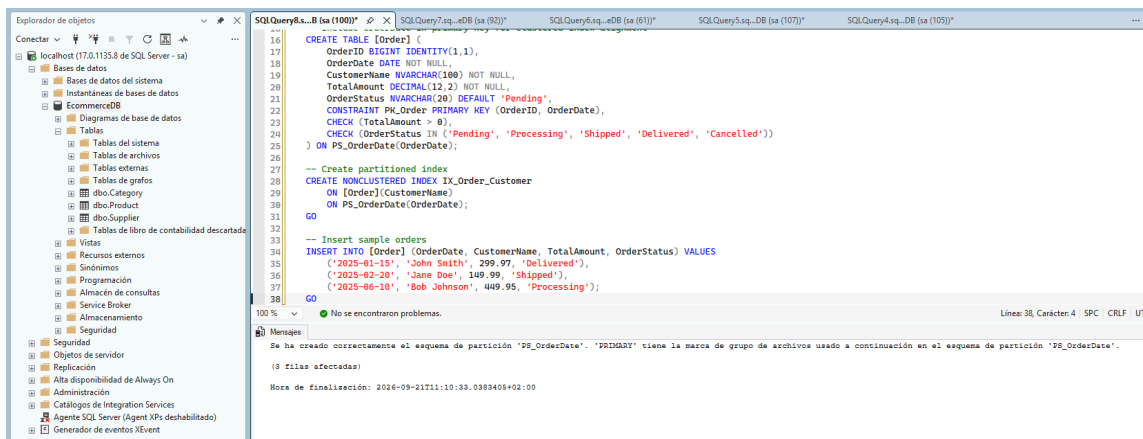
-- 1. Función de partición
CREATE PARTITION FUNCTION PF_OrderDate (DATE)
AS RANGE RIGHT FOR VALUES ('2025-01-01', '2025-04-01', '2025-07-01', '2025-10-01');

-- 2. Esquema de partición
CREATE PARTITION SCHEME PS_OrderDate
AS PARTITION PF_OrderDate ALL TO ([PRIMARY]);
```

```
-- 3. Tabla Order particionada
CREATE TABLE [Order] (
    OrderID BIGINT IDENTITY(1,1),
    OrderDate DATE NOT NULL,
    CustomerName NVARCHAR(100) NOT NULL,
    TotalAmount DECIMAL(12,2) NOT NULL,
    OrderStatus NVARCHAR(20) DEFAULT 'Pending',
    CONSTRAINT PK_Order PRIMARY KEY (OrderID, OrderDate),
    CHECK (TotalAmount > 0),
    CHECK (OrderStatus IN ('Pending', 'Processing', 'Shipped', 'Delivered', 'Cancelled'))
) ON PS_OrderDate(OrderDate);

-- 4. Índice particionado
CREATE NONCLUSTERED INDEX IX_Order_Customer
    ON [Order](CustomerName)
    ON PS_OrderDate(OrderDate);
GO

-- Inserción de pedidos de muestra
INSERT INTO [Order] (OrderDate, CustomerName, TotalAmount, OrderStatus) VALUES
    ('2025-01-15', 'John Smith', 299.97, 'Delivered'),
    ('2025-02-20', 'Jane Doe', 149.99, 'Shipped'),
    ('2025-06-10', 'Bob Johnson', 449.95, 'Processing');
GO
```



Verificamos la distribución de las filas en cada partición física utilizando la función del sistema `$PARTITION` :

```
USE EcommerceDB;
GO

SELECT $PARTITION.PF_OrderDate(OrderDate) AS PartitionNumber,
    COUNT(*) AS OrdersInPartition,
    MIN(OrderDate) AS MinDate,
    MAX(OrderDate) AS MaxDate
FROM [Order]
GROUP BY $PARTITION.PF_OrderDate(OrderDate);
```

The screenshot shows the SQL Server Enterprise Manager interface on the left, displaying the 'EcommerceDB' database structure. The right pane shows a SQL query window with the following code:

```

1  USE EcommerceDB;
2  GO
3
4  -- Query by partition
5  SELECT
6      $PARTITION.PF_OrderDate(OrderDate) AS PartitionNumber,
7      COUNT(*) AS OrdersInPartition,
8      MIN(OrderDate) AS MinDate,
9      MAX(OrderDate) AS MaxDate
10 FROM [Order]
11 GROUP BY $PARTITION.PF_OrderDate(OrderDate);

```

Below the query, the 'Results' tab shows the following data:

PartitionNumber	OrdersInPartition	MinDate	MaxDate
1	2	2025-01-15	2025-02-20
2	3	2025-06-10	2025-06-10

Los dos pedidos de enero y febrero quedan asignados a la partición 2, mientras que el de junio reside en la partición 3.

7. Detalle de Pedidos con Secuencia (Create order details with SEQUENCE)

A diferencia de `IDENTITY`, los objetos `SEQUENCE` generan números secuenciales de forma independiente a cualquier tabla.

Creamos la secuencia `OrderLineSequence` y la tabla `OrderDetail`, que incluye una columna calculada (`LineTotal`) que multiplica la cantidad por el precio unitario:

```

USE EcommerceDB;
GO

-- Objeto SEQUENCE para líneas de pedido
CREATE SEQUENCE OrderLineSequence START WITH 1 INCREMENT BY 1;

-- Tabla OrderDetail
CREATE TABLE OrderDetail (
    OrderLineID INT PRIMARY KEY,
    OrderID BIGINT NOT NULL,
    OrderDate DATE NOT NULL,
    ProductID INT NOT NULL,
    Quantity INT NOT NULL,
    UnitPrice DECIMAL(10,2) NOT NULL,
    LineTotal AS (Quantity * UnitPrice),

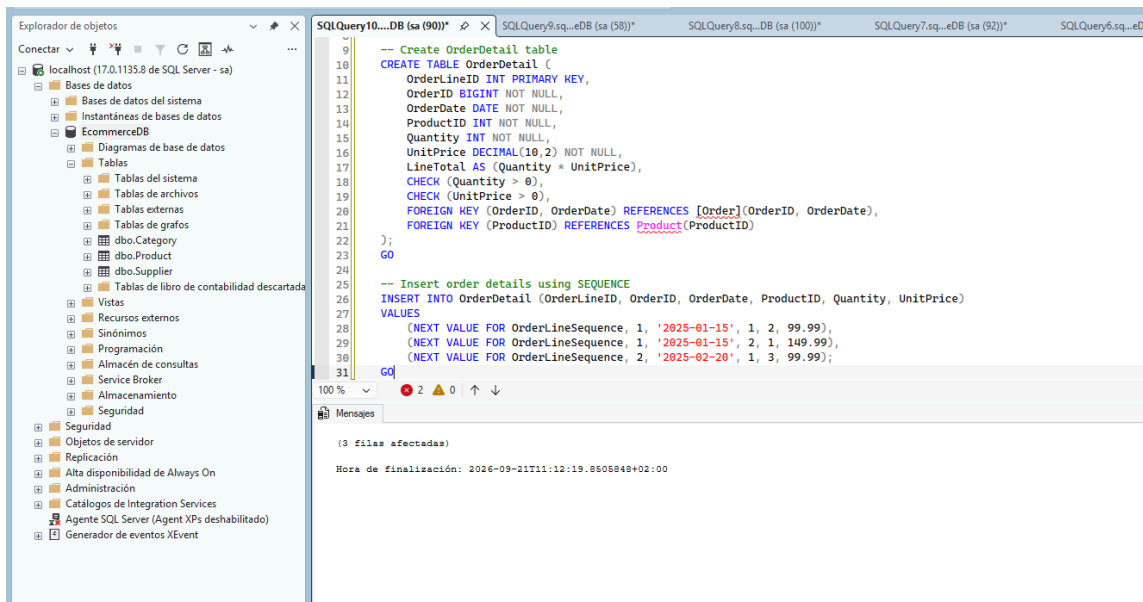
```

```

CHECK (Quantity > 0),
CHECK (UnitPrice > 0),
FOREIGN KEY (OrderID, OrderDate) REFERENCES [Order](OrderID, OrderDate),
FOREIGN KEY (ProductID) REFERENCES Product(ProductID)
);
GO

-- Inserción utilizando NEXT VALUE FOR
INSERT INTO OrderDetail (OrderLineID, OrderID, OrderDate, ProductID, Quantity, UnitPrice)
VALUES
    (NEXT VALUE FOR OrderLineSequence, 1, '2025-01-15', 1, 2, 99.99),
    (NEXT VALUE FOR OrderLineSequence, 1, '2025-01-15', 2, 1, 149.99),
    (NEXT VALUE FOR OrderLineSequence, 2, '2025-02-20', 1, 3, 99.99);
GO

```



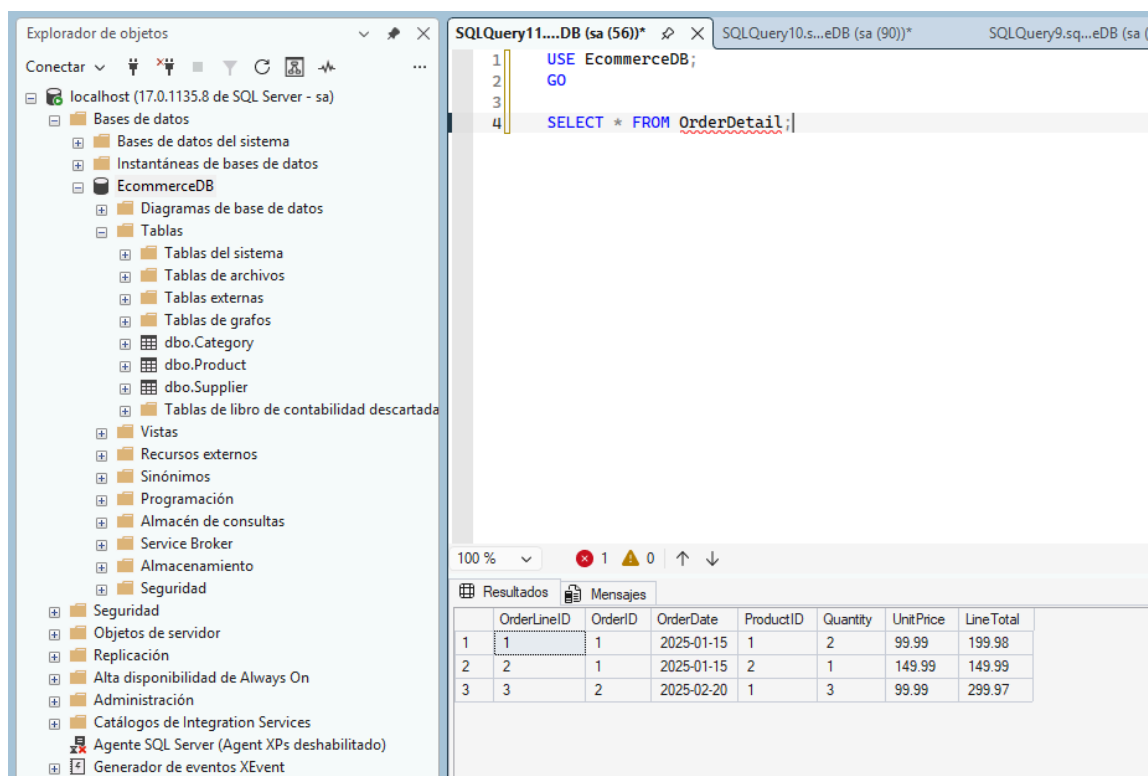
Consultamos los registros insertados:

```

USE EcommerceDB;
GO

SELECT * FROM OrderDetail;

```



8. Verificación de Objetos de Base de Datos (Verify database objects)

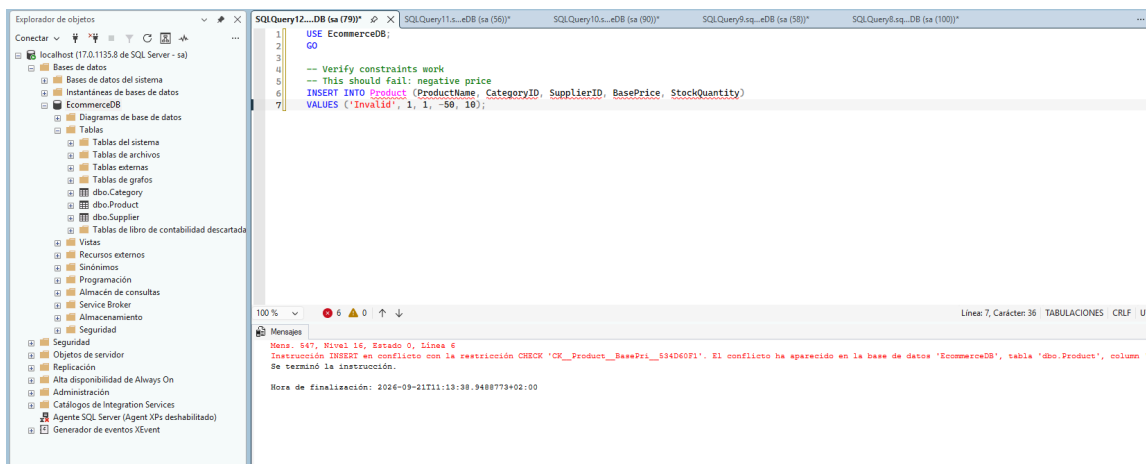
Prueba de restricciones de integridad

Para confirmar que las restricciones **CHECK** protegen adecuadamente el sistema, intentamos deliberadamente insertar un producto con un precio negativo:

```
USE EcommerceDB;
GO
```

```
-- Esta inserción debe fallar
```

```
INSERT INTO Product (ProductName, CategoryID, SupplierID, BasePrice, StockQuantity)
VALUES ('Invalid', 1, 1, -50, 10);
```



SQL Server devuelve el error Mens. 547, Nivel 16 , confirmando que el conflicto con la restricción CHECK impidió la inserción de datos no válidos.

Verificación conjunta de JSON, particionado y tablas temporales

Ejecutamos una consulta combinada para validar los tres componentes avanzados del laboratorio:

```
USE EcommerceDB;
GO

-- 1. Verificación de consultas sobre JSON
SELECT ProductName, JSON_VALUE(Metadata, '$.color') AS Color
FROM Product
WHERE Metadata IS NOT NULL;

-- 2. Verificación del particionado
SELECT $PARTITION.PF_OrderDate(OrderDate) AS Partition, COUNT(*) AS RecordCount
FROM [Order]
GROUP BY $PARTITION.PF_OrderDate(OrderDate);

-- 3. Verificación de la tabla temporal
SELECT ProductID, CurrentPrice, SysStartTime, SysEndTime
FROM ProductPrice FOR SYSTEM_TIME ALL
ORDER BY ProductID, SysStartTime;
```

The screenshot displays the SQL Server Enterprise Manager interface on the left, showing the 'Bases de datos' (Databases) folder expanded to 'EcommerceDB'. The 'Tablas' (Tables) folder is also expanded, showing tables like 'dbo.Product', 'dbo.ProductPrice', and 'dbo.Supplier'. The main window shows a SQL query in the 'SQLQuery13...eDB (sa (91))' window. The query is as follows:

```
1 USE EcommerceDB;
2 GO
3
4 -- Verify JSON queries work
5 SELECT ProductName, JSON_VALUE(Metadata, '$.color') AS Color
6 FROM Product
7 WHERE Metadata IS NOT NULL;
8
9 -- Verify partitioning
10 SELECT $PARTITION.PF_OrderDate(OrderDate) AS Partition, COUNT(*) AS RecordCount
11 FROM [Order]
12 GROUP BY $PARTITION.PF_OrderDate(OrderDate);
13
14 -- Verify temporal table
15 SELECT ProductID, CurrentPrice, SysStartTime, SysEndTime
16 FROM ProductPrice FOR SYSTEM_TIME ALL
17 ORDER BY ProductID, SysStartTime;
```

The results pane shows three tables of data:

ProductName	Color
Wireless Mouse	blue
Cotton T-Shirt	red

Partition	RecordCount
1	2
2	1

ProductID	CurrentPrice	SysStartTime	SysEndTime
1	99.99	2026-09-21 08:55:38.7657312	2026-09-21 08:55:38.7666951
2	109.99	2026-09-21 08:55:38.7666951	9999-12-31 23:59:59.9999999
3	149.99	2026-09-21 08:55:38.7657312	9999-12-31 23:59:59.9999999

Los tres paneles confirman que los atributos JSON se leen correctamente, que los pedidos están distribuidos en sus particiones y que el histórico de precios conserva su trazabilidad.

9. Limpieza de Recursos (Cleanup)

Una vez completadas y validadas las tareas del laboratorio, los objetos creados pueden eliminarse de la instancia mediante los siguientes pasos en SSMS:

1. En el Explorador de objetos, desplegar la carpeta **Bases de datos**.
2. Hacer clic con el botón derecho sobre **EcommerceDB** y seleccionar **Eliminar**.
3. En el cuadro de diálogo, marcar la casilla **Cerrar las conexiones existentes**.
4. Seleccionar **Aceptar**.

Conclusiones

En este ejercicio se ha implementado un esquema completo y robusto para una plataforma de comercio electrónico en SQL Server, integrando:

1. **Restricciones declarativas:** Aseguran la validez de los datos (precios y cantidades positivas, unicidad de nombres) desde el propio motor.
2. **Versionado temporal del sistema:** Registra de manera transparente cualquier alteración en datos críticos como los precios.
3. **Soporte nativo para JSON:** Proporciona la flexibilidad de esquemas no estructurados con la capacidad de indexación del modelo relacional.

4. **Particionamiento por rangos:** Prepara la tabla de pedidos para operar eficientemente ante grandes volúmenes de información.
5. **Secuencias independientes:** Desacoplan la generación de claves de tablas específicas para mayor flexibilidad.